Security

Security in Distributed Systems

- Security threats:
  - Unauthorized information disclosure.
  - Unauthorized information modification.
  - Unauthorized denial of use.
- A security policy describes which actions the entities (users, services, data, machines, etc) in a system are allowed to take and which ones are prohibited.
- A policy is enforced using a security mechanism. Four important security mechanisms include:
  - Encryption.
  - Authentication.
  - Authorization.
  - Monitoring and auditing.

Cryptography

- Symmetric versus asymmetric cryptography. In symmetric the encryption and decryption keys are the same while in asymmetric cryptography they are different.
- Public key cryptography – asymmetric.
- RSA (Rivest, Shamir, and Adleman) scheme – asymmetric.
- Examples: Secure shell (`ssh, slogin, sshd`), PGP (Pretty Good Privacy, a free cryptographic package), Kerberos network authentication, SSL (Secure Sockets Layer, used with https protocol).

"If privacy is outlawed, only outlaws will have privacy." Zimmerman (author of PGP)

Public key cryptography

Each participant has a *public key* and a *secret key*. In RSA public-key cryptosystem, each key consists of a pair of large integers.
Alice has key (P_A, S_A).
Bob has key (P_B, S_B).
Let $\mathcal{D}$ be the set of permissible messages. Then we require the following conditions.

$$P_A, S_A, P_B, S_B : \mathcal{D} \to \mathcal{D}$$

$$M = S_A(P_A(M))$$

$$M = P_A(S_A(M))$$

$$M = S_B(P_B(M))$$

$$M = P_B(S_B(M))$$

Sending an encrypted message

1. Bob obtains Alice's public key P_A.
2. Bob computes the ciphertext $C = P_A(M)$ corresponding to the message M and sends C to Alice.
3. When Alice receives the ciphertext C, she applies her secret key S_A to retrieve the original message: $M = S_A(C)$.

Digital signature

1. Alice computes her digital signature $\sigma = S_A(M^{'})$ for the message $M^{'}$ using her secret key.
2. Alice sends the message/signature pair $(M^{'}\sigma)$ to Bob.
3. When Bob receives $(M^{'}, \sigma)$, he can verify that it originated from Alice by using Alice's public key to verify that $M^{'} = P_A(\sigma)$.

A digital signature is verifiable by anyone who has access to the signers public key. The signed message is not encrypted.

Encrypted and signed message

1. Alice computes her digital signature $\sigma = S_A(M^{'})$ for the message $M^{'}$ using her secret key.
2. Alice appends her digital signature to the message and then encrypts the resulting pair with Bob's public key to send $P_B(M^{'}\sigma)$.
3. Bob decrypts the message using his secret key.
4. Bob verifies Alice's signature using her public key.

More on cryptography

- The security of the public-key cryptosystem rests in large part on the difficulty of factoring large integers.
  - If factoring large integers is easy, then breaking the RSA cryptosystem is easy.
  - If factoring large integers is hard, then whether breaking RSA is hard is an unproven statement. However decades of research has not found an easy way to break the RSA system.
  - In theory, Quantum Computers can break RSA, but they are not yet practical. Increasing the key length can make it more difficult to break RSA even with quantum computers. The current recommendation is to use a key length of at least 2048 bits.
- A perfect tool for electronic contracts, electronic checks, e-cash, etc. However cryptography is not a panacea for every security issue.
- How do you get your public key in the beginning. Get a certificate from a trusted authority.
- Public-key cryptosystem involve multiple-precision arithmetic which is considerably slower. Most practical systems use a hybrid approach.

Encryption Algorithms in Java

- The packages `javax.crypto` and `java.security` provide the basic mechanisms.
- We can get a list of all available security providers with some simple code. See example: security/providers/ListProviders.java
- See code snippet below:

```
import java.security.Provider;
import java.security.Provider.Service;
import java.security.Security;
import java.util.Set;

public class ListProviders {
    public static void main (String[] args) {
        Provider[] list = Security.getProviders();
        for (Provider e: list) {
            System.out.println(e);
            Set<Service> serviceList = e.getServices();
            for (Service s: serviceList)
                System.out.println("\t" + s);
            System.out.println();
        }
    }
}
```

Symmetric Encryption in Java

- Sample code that generates a key and uses it to encrypt information and then stores both the key and the encrypted information on the disk. See examples: EncryptTest.java DecryptTest.java
- Uses the Advanced Encryption Standard (AES)
- Code snippet:

```
// security/encryption/EncryptTest.java
FileOutputStream dataFile = new FileOutputStream("data.encrypted");
ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream("key");

KeyGenerator kg = KeyGenerator.getInstance("AES");
Key key = kg.generateKey();
oos.writeObject(key);

Cipher cipher = Cipher.getInstance("AES");
byte[] data = "Hello World!".getBytes();
cipher.init(Cipher.ENCRYPT_MODE, key);
byte[] result = cipher.doFinal(data);
dataFile.write(data);
```

- **Beware!** The key is not protected. It must at least not be readable by anyone else.

Symmetric Decryption in Java

- Sample code that reads in a key and uses it to decrypt information from an encrypted file.

```
// security/encryption/DecryptTest.java
File dataFile = new File("data.encrypted");
FileInputStream data = new FileInputStream(dataFile);
ObjectInputStream ois = new ObjectInputStream( new FileInputStream("key"));

Key key = (Key) ois.readObject();
Cipher cipher = Cipher.getInstance("AES");
byte [] result = new byte[(int)dataFile.length()];
int n = data.read(result);
cipher.init(Cipher.DECRYPT_MODE, key);
byte[] original = cipher.doFinal(result);
```

One Way Hash Functions

- A one-way hash function is a function that takes an input and produces a fixed-size string of bytes. The output is typically a digest that is unique to the input data. It is designed to be irreversible, meaning that it should be computationally infeasible to reverse the function and obtain the original input from the output.
- Examples: MD5 (Message Digest 5), SHA-1 (Secure Hash Algorithm 1), SHA-256, SHA-512, etc.
- See example securehash/SHA2Test.java on how to use MD5 sums and Secure hash functions in Java.
  - Please note that MD5 sums have a known flaw so research them fully before relying on them.
- Another issue is handling password input without showing input. See the example PasswordTest.java in the folder `security/passwordinput` for more details.

Secure Communication

- Secure communication is a critical aspect of distributed systems, as it ensures that data transmitted between parties remains confidential and protected from unauthorized access.
- Secure Sockets Layer (SSL) and its successor, Transport Layer Security (TLS), are widely used protocols for securing communication over the internet. They provide encryption, authentication, and data integrity.
- RMI (Remote Method Invocation) can be secured using SSL/TLS.

Secure Sockets Layer

- Secure Sockets Layer (SSL). The most widely used protocol used for SSL provides privacy, data integrity, authenticity and non-repudiation.
  - Uses asymmetric key cryptography (public/private key) to authenticate the identities of the communicating parties.
  - Uses asymmetric key cryptography (public/private key) to encrypt the shared encryption key that is used during the SSL session.
  - Uses symmetric key cryptography for data encryption between client and server.
- RMI with SSL. Java has two classes that provide the ability to secure the communication channel using SSL/TLS (Secure Socket Layer/Transport Layer Security) protocols:

  `javax.rmi.ssl.SslRMIClientSocketFactory`
  `javax.rmi.ssl.SslRMIServerSocketFactory`

Overview of RMI with SSL

- *Establish a SSL session using a SSL handshake*. The client initiates a connection and the server responds. This a multi-step process. The net result is that the client and server agree on an encryption scheme.
- *The server sends its certificate and the client verifies it*. The certificate sent by the server comes from the server's keystore as a database for the contents. The client either verifies the server's signature by either trusting the server or trusting one of the signers in the certificate chain provided by the server.
- *The client stores certificates in its truststore*. The default truststore is `cacerts`, which can be found in the following location: `$JAVA_HOME/jre/lib/security/cacerts`. Check the listed authorities in it with the command (default password is *changeit*): keytool -list -v -keystore cacerts
- *Next the client uses the public key of the server to send a `ClientKeyExchange` message to the server*. The message contains some random information that is used to generate a symmetric key that will be used for encrypting the content during the data exchange.
- *Next the client sends a `ChangeCipherSpec` message indicating that it is ready to communicate*. This message is followed by a `Finished` message.
- *The server responds by sending its `ChangeCipherSpec` message and a `Finished` message*.

Example SSL Handshake

```
*** ClientHello, TLSv1
RandomCookie:  GMT: 1141769969 bytes = { 93, 99, 48, 178, 50, 21, 255,
207, 135, 20, 150, 233, 207, 151, 26, 126, 200, 93, 146, 59, 53, 232,
2, 209, 238, 34, 219, 178 }
Session ID:  {}
Cipher Suites: [SSL_RSA_WITH_RC4_128_MD5, SSL_RSA_WITH_RC4_128_SHA,
TLS_RSA_WITH_AES_128_CBC_SHA, TLS_DHE_RSA_WITH_AES_128_CBC_SHA,
TLS_DHE_DSS_WITH_AES_128_CBC_SHA, SSL_RSA_WITH_3DES_EDE_CBC_SHA,
SSL_DHE_RSA_WITH_3DES_EDE_CBC_SHA, SSL_DHE_DSS_WITH_3DES_EDE_CBC_SHA,
SSL_RSA_WITH_DES_CBC_SHA, SSL_DHE_RSA_WITH_DES_CBC_SHA,
SSL_DHE_DSS_WITH_DES_CBC_SHA, SSL_RSA_EXPORT_WITH_RC4_40_MD5,
SSL_RSA_EXPORT_WITH_DES40_CBC_SHA, SSL_DHE_RSA_EXPORT_WITH_DES40_CBC_SHA,
SSL_DHE_DSS_EXPORT_WITH_DES40_CBC_SHA]
Compression Methods:  { 0 }
***

*** ServerHello, TLSv1
RandomCookie:  GMT: 1141769970 bytes = { 214, 236, 161, 51, 175, 144,
66, 122, 86, 62, 242, 54, 229, 209, 121, 18, 164, 196, 77, 233, 16, 174,
20, 9, 92, 153, 236, 197 }
Session ID:  {68, 14, 7, 242, 21, 148, 20, 218, 17, 110, 197, 208, 17,
91, 178, 156, 22, 52, 57, 41, 20, 215, 6, 80, 62, 112, 182, 111, 31,
35, 51, 164}
Cipher Suite: SSL_RSA_WITH_RC4_128_MD5
Compression Method: 0
***
```

Example SSL Handshake (continued)

```
From Client
*** ClientKeyExchange, RSA PreMasterSecret, TLSv1
Random Secret:  { 3, 1, 155, 121, 164, 80, 202, 181, 110, 118, 28, 78,
85, 173, 230, 166, 234, 188, 171, 204, 130, 167, 6, 155, 155, 178, 70,
20, 88, 244, 141, 220, 177, 167, 4, 147, 24, 129, 165, 171, 70, 23, 132,
74, 144, 20, 156, 227 }

From Client
main, WRITE: TLSv1 Change Cipher Spec, length = 1
[Raw write]: length = 6
0000: 14 03 01 00 01 01                                  ......
*** Finished

From Server
RMI TCP Connection(2)-132.178.248.51, WRITE: TLSv1 Change Cipher Spec, length =
[Raw write]: length = 6
0000: 14 03 01 00 01 01                                  ......
*** Finished
```

Generated using the `-Djavax.net.debug=all` option to the java VM. Also, see the option `-Djavax.net.ssl.debug=all` for more specific SSL debugging information.

Creating the setup

- Generate a keystore that has a key pair (public and private key) along with a self-signed certificate. The store type is PKCS12, which is an industry standard.
  `keytool -deststoretype pkcs12 -genkey -alias SecureServer -keyalg RSA -keystore Server_Keystore`
- Examine the contents of the generated Server Keystore.
  keytool -list -v -keystore Server_Keystore
- Create a self-signed certificate.
  keytool -export -alias SecureServer -keystore Server_Keystore -rfc -file Server.cer
- To see what the certificate looks like.
  cat Server.cer
- Next we import the server certificate into a truststore that can be used by the client.
  keytool -import -alias SecureServer -file Server.cer -keystore Client_Truststore
- To verify the contents of the truststore that we created, we issue the following command.
  keytool -list -v -keystore Client_Truststore

In our example, we are working with a self-signed certificate instead of certificates signed by Certification Authority (CA). If there is a need to get the certificate signed by a CA then a Certificate Signing Request(CSR) needs to be generated. The generated CSR, then, should to be submitted along with other pertinent information to a Certification Authority such as VeriSign or USPS, who will then digitally sign the certificate.

Using SSL Sockets

- See example in the folder security/sslsockets.
- On the server side.

```
import javax.net.ssl.*;
System.setProperty("javax.net.ssl.keyStore", "Server_KeyStore");
System.setProperty("javax.net.ssl.keyStorePassword", "password");
SSLServerSocketFactory sslSrvFact =
        (SSLServerSocketFactory) SSLServerSocketFactory.getDefault();
SSLServerSocket ss = (SSLServerSocket) sslSrvFact.createServerSocket(port);
SSLSocket sock = (SSLSocket) ss.accept();
```

- On the client side.

```
import javax.net.ssl.*;
System.setProperty("javax.net.ssl.trustStore", "Client_Truststore");
System.setProperty("javax.net.ssl.trustStorePassword", "password");
SSLSocketFactory sslFact = (SSLSocketFactory)SSLSocketFactory.getDefault();
SSLSocket server = (SSLSocket)sslFact.createSocket(host, port);
```

- Make sure to adjust path to Server and Client truststore based on your project layout!

Using RMI with SSL Sockets

- See examples in the folder security/rmisslex1
- On the server side.

```
System.setProperty("javax.net.ssl.keyStore","./resources/Server_Keystore");
System.setProperty("javax.net.ssl.keyStorePassword","password");
RMIClientSocketFactory rmiClientSocketFactory =
                                        new SslRMIClientSocketFactory();
RMIServerSocketFactory rmiServerSockeyFactory =
                                        new SslRMIServerSocketFactory();
DateServer server = (DateServer) UnicastRemoteObject.exportObject(this, 0,
                          rmiClientSocketFactory, rmiServerSockeyFactory);
Registry registry = LocateRegistry.createRegistry(port);
registry.rebind(name, server);
```

- On the client side.

```
System.setProperty("javax.net.ssl.trustStore", "Client_Truststore");
Registry reg = LocateRegistry.getRegistry(host, port);
DateServer server = (DateServer) reg.lookup("DateServerImpl");
```

- Also see example in folder security/rmisslex2

Key Management

- **Key management** is the process of generating, establishing, distributing, storing, and revoking cryptographic keys. It is a critical aspect of any secure system, as the security of the system relies heavily on the secrecy and integrity of the keys.
- **Key establishment** is the process of securely sharing keys between parties. This can be done using various methods, such as Diffie-Hellman key exchange, RSA key exchange, or using a trusted third party (e.g., a Key Distribution Center or a Public Key Infrastructure).
- **Key distribution** is the process of securely distributing keys to the parties that need them. This can be done using various methods, such as using a secure channel (e.g., SSL/TLS), using a trusted third party, or using a key agreement protocol.

SSH Public Key Authentication (Including Setup)

1. **Key Generation (Setup):** The client generates a public/private key pair (e.g., `ssh-keygen`). The private key is kept secret; the public key can be shared.
2. **Key Distribution (Setup):** The client's public key is copied to the server (e.g., `authorized_keys` file).
3. **Connection Initiation:** The client connects to the SSH server and indicates it wants to authenticate using a public key.
4. **Challenge-Response:** The server sends a challenge. The client signs the challenge using its private key.
5. **Verification and Access:** The server verifies the signature using the stored public key. If valid, authentication succeeds and access is granted.

SSH Certificate-Based Authentication: Steps

1. **Certificate Issuance:** A Certificate Authority (CA) signs the user's public key, creating an SSH certificate.
2. **Client Initiates Connection:** The client connects to the SSH server and presents its public key along with the certificate.
3. **Certificate Verification:** The server verifies that the certificate is signed by a trusted CA and checks validity constraints (time, identity, permissions).
4. **Challenge-Response:** The server sends a challenge; the client signs it using its private key.
5. **Authentication Success:** The server verifies the signature using the public key from the certificate and grants access if valid.

SSH Authentication: Public Key vs Certificate-Based

Feature	**Public Key**	**SSH Certificate**
Trust Model	Per-server trust	Central Certificate Authority (CA)
Scalability	Poor (manual key distribution)	Excellent (CA-based trust)
Key Management	Difficult to manage at scale	Centralized and simpler
Revocation	Manual removal of keys	Easy (use short-lived certs)
Flexibility	Limited	Supports constraints (time, IP, roles)
Deployment	Copy keys to each server	Configure server to trust CA
Security Model	Static trust	Dynamic, policy-driven trust

Diffie–Hellman Key Exchange (Idea)

Goal: Establish a shared secret over an insecure channel

Public parameters:

- Large prime p
- Generator g

Key exchange steps:

- Client chooses secret a, sends:

$$A = g^a \bmod p$$

- Server chooses secret b, sends:

$$B = g^b \bmod p$$

Key insight

Even though A and B are public, a and b remain secret.

Diffie–Hellman Key Exchange (Shared Secret)

Compute shared key:

- Client computes:

$$K = B^a \bmod p = g^{ba} \bmod p$$

- Server computes:

$$K = A^b \bmod p = g^{ab} \bmod p$$

Result:

$$\text{Shared secret } K = g^{ab} \bmod p$$

Security:

- Attacker sees g, p, A, B
- Cannot efficiently compute g^{ab} (Discrete Log problem)

Use

K is used to derive encryption keys for secure communication.

Authentication

Authentication is the process of verifying the identity of a user, device, or system.

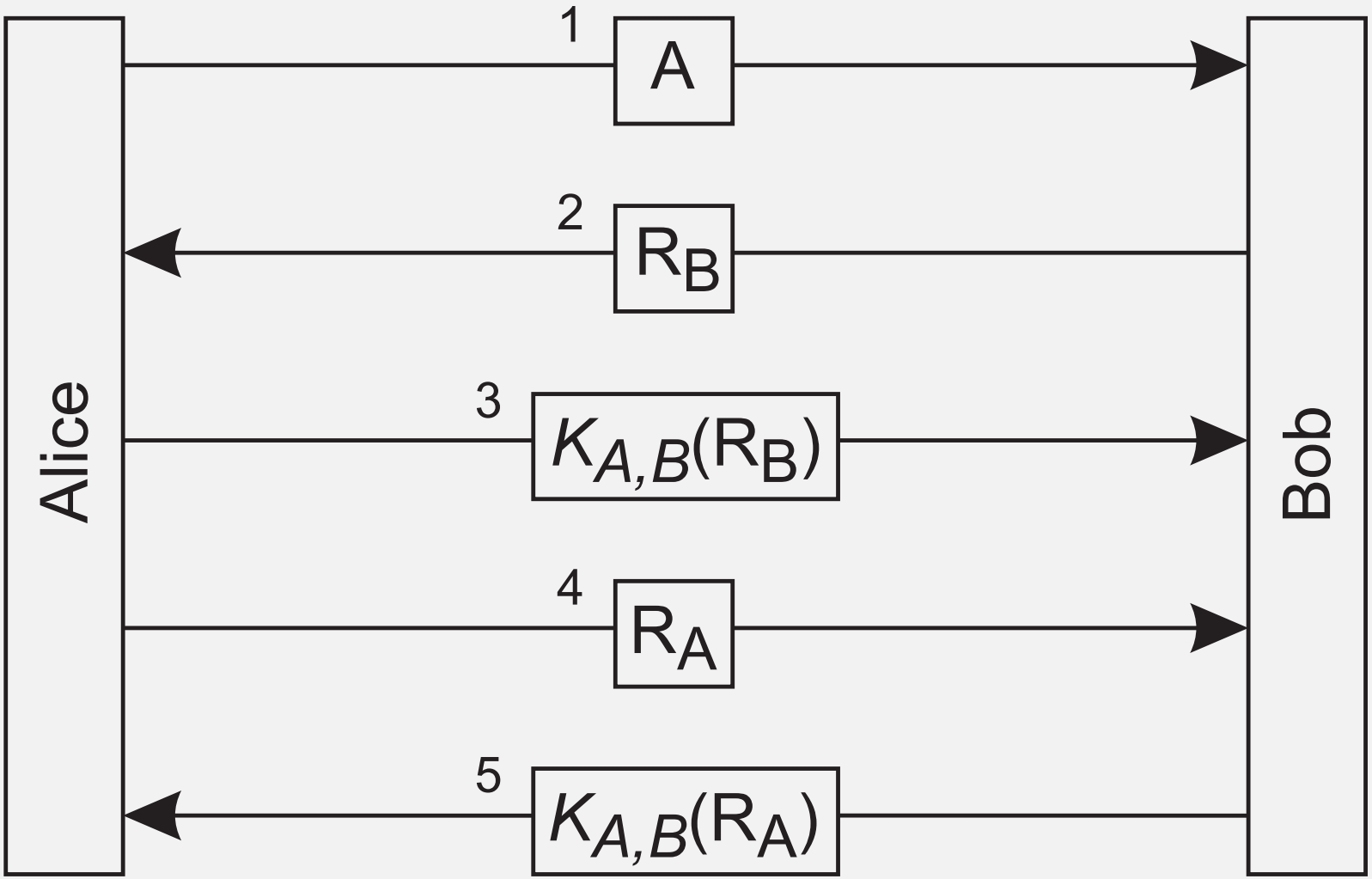
- Based on what a client **knows**, such as a password or a PIN.
- Based on what a client **has**, such as an ID card, cell phone or a software security token or a smart card.
- Based on what a client **is**, such as static biometric data (facial characteristics, fingerprint, retina scan, etc).
- Based on what a client **does**, i.e. dynamic biometrics (voice patterns, typing rhythm, etc).

Single-Factor Authentication (SFA) relies on one of the above methods, such as a password or a security token (e.g. used in SSH key-based authentication).
Multi-Factor Authentication (MFA) combines two or more of the above methods to enhance security.
Continuous Authentication is an emerging approach that continuously verifies the identity of a user or device throughout a session, rather than just at the initial login.

Authentication Protocols

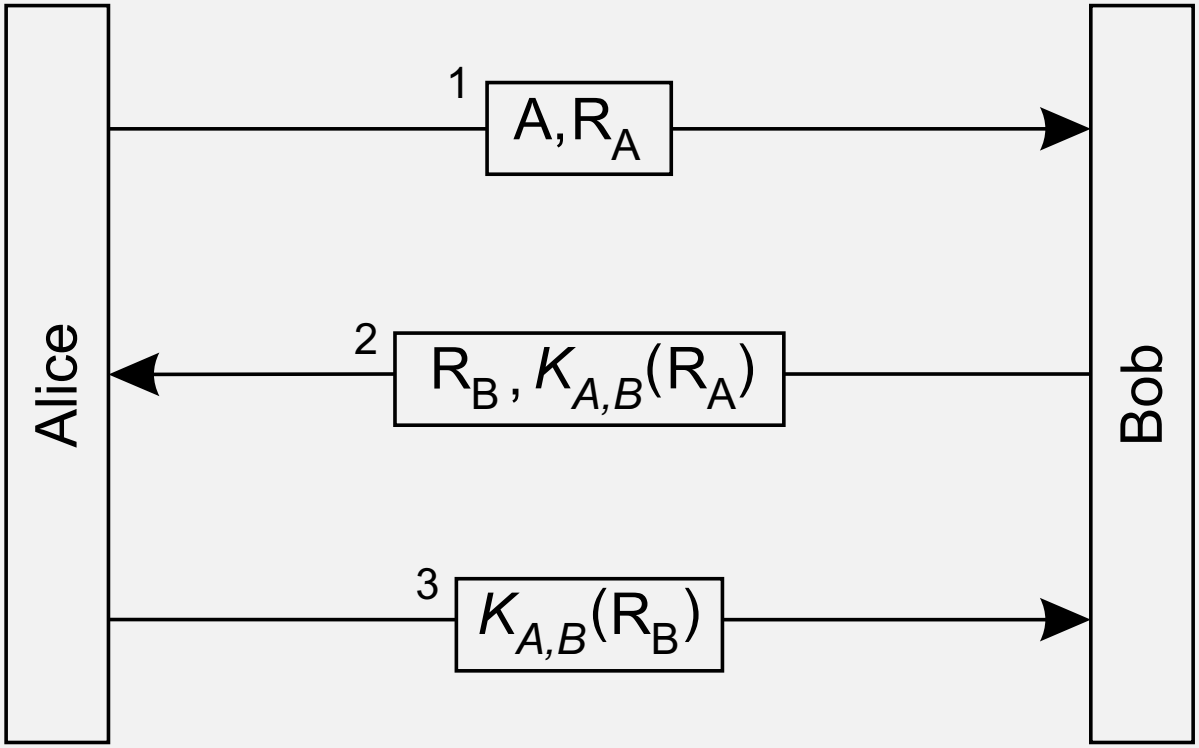
- **Using a shared secret key**
- **Using a key distribution center (KDC)**
- **Using a public key infrastructure (PKI)**

Shared Secret Key Authentication



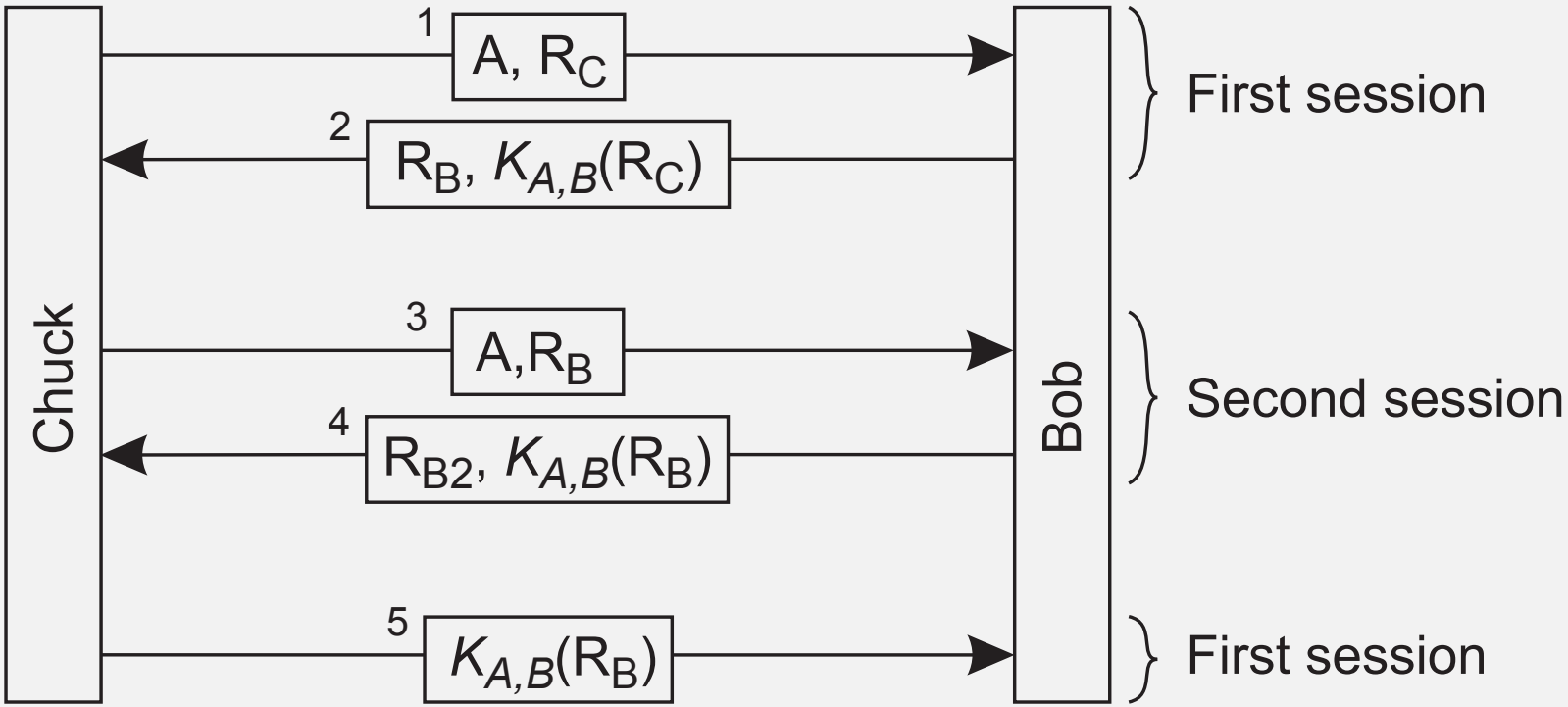
- The client and server have shared a secret key $K_{A,B}$ that was exchanged out-of-band prior to this protocol.

Difficulty of Designing Security Protocols



An unfortunate "optimization" of the shared secret key authentication protocol leads to a security flaw. The client sends the password in plaintext to the server, which is vulnerable to eavesdropping and man-in-the-middle attacks.

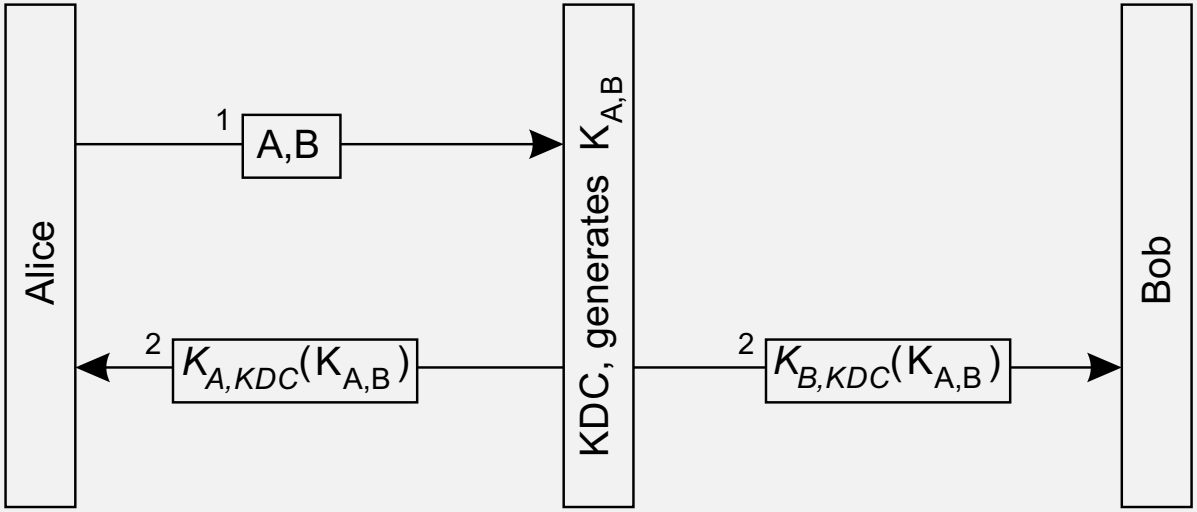
Difficulty of Designing Security Protocols

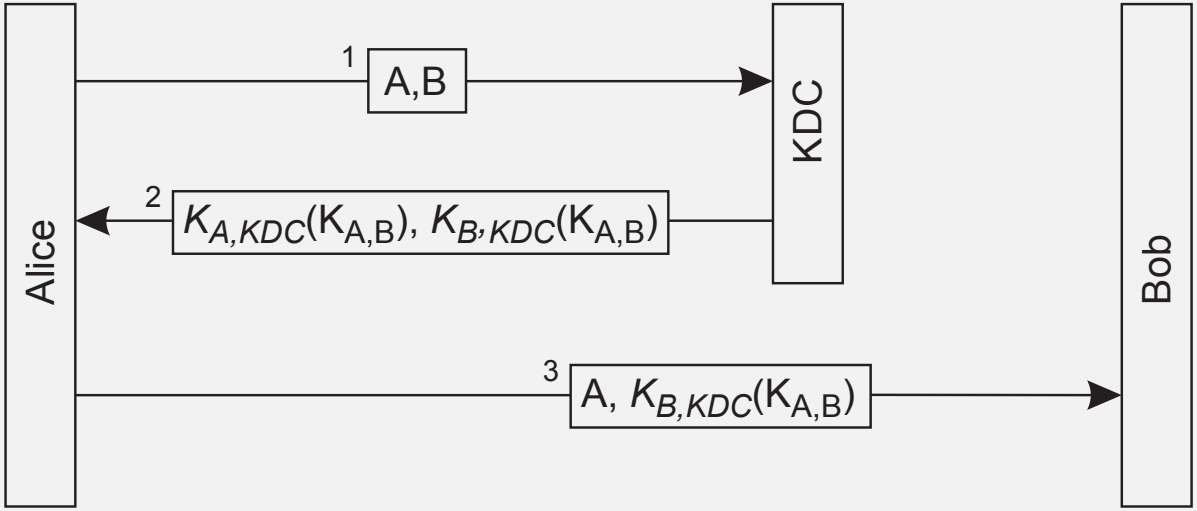


Key Distribution Center Authentication (1)

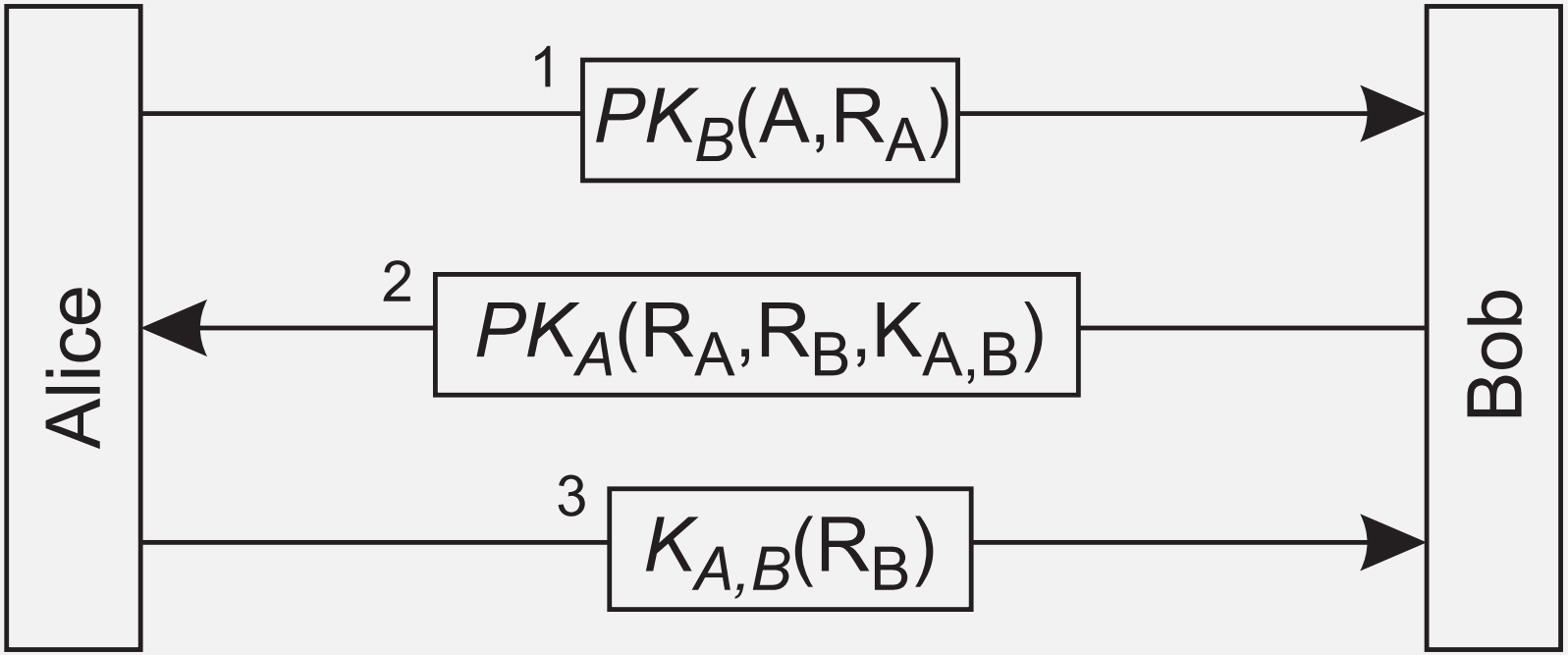
- The previous solution requires the client and server to share a secret key. If we have n hosts as it would require $n(n-1)/2$ keys, which is not practical for large n.
- A Key Distribution Center (KDC) shares a secret key with each host and can be used to facilitate authentication without requiring direct key sharing between clients and servers. This reduces the number of keys needed to n.

Key Distribution Center Authentication (2)





Public Key Infrastructure Authentication (1)



Public Key Infrastructure Authentication (2)

- After authentication, we use a session specific symmetric key for encrypting the communication between the client and server.
- Symmetric key encryption is much faster than public key encryption
- Since it is discarded after the session, it does not pose a long-term security risk.

Securely Sharing GitHub SSH Keys to EC2

A case study of securely sharing GitHub SSH keys to EC2 instances, comparing three methods:

- **SSH Agent Forwarding**
- **GitHub Deploy Keys**
- **AWS Secrets Manager**

Comparison of GitHub SSH Sharing Methods

Method	Key Location	Best For	Security
Agent Forwarding	Local Machine	Interactive work	High
Deploy Keys	EC2 Instance	Single Repo	Medium
Secrets Manager	AWS (Encrypted)	Enterprise Fleets	Highest

Table: Secure access options for GitHub to EC2

- **High:** Private key never leaves your local hardware.
- **Highest:** Centralized encryption with IAM audit logs.

Scenario 1: SSH Agent Forwarding

Best For: Individual developers and interactive debugging.

- **Mechanism:** Your local private key is "forwarded" to the EC2 session without ever being stored on the remote disk.
- **Pros:** Highest personal security; key never leaves your machine.
- **Cons:** Requires an active session; doesn't work for background tasks.

```
# 1. Add key to local agent
ssh-add ~/.ssh/id_ed25519

# 2. Connect to EC2 with -A flag
ssh -A ec2-user@your-ec2-ip

# 3. Test GitHub access on EC2
ssh -T git@github.com
```

Scenario 2: GitHub Deploy Keys

Best For: Automated builds or single-purpose application servers.

- **Mechanism:** A unique key pair generated on the EC2 instance is granted read-only access to a specific repository.
- **Pros:** Access is scoped to one repo; works for background cron jobs.
- **Cons:** Private key resides on the EC2 disk; needs management per repo.

```
# 1. Generate key on EC2
ssh-keygen -t ed25519 -f ~/.ssh/github_deploy

# 2. Add public key to GitHub Repo Settings -> Deploy
    Keys
cat ~/.ssh/github_deploy.pub

# 3. Use the key for git operations
git clone git@github.com:user/repo.git
```

Scenario 3: AWS Secrets Manager

Best For: Enterprise production fleets and auto-scaling groups.

- **Mechanism:** Keys are stored in an encrypted vault. Instances fetch them via IAM roles during bootstrap.
- **Pros:** Centralized rotation, audited access, and no keys in AMIs.
- **Cons:** Requires AWS CLI and IAM configuration.

```
# Fetch the secret via AWS CLI
aws secretsmanager get-secret-value \
    --secret-id MyGitHubPrivateKey \
    --query SecretString --output text > ~/.ssh/id_rsa

chmod 400 ~/.ssh/id_rsa
```

Secure Password Handling Model (1)

- Send the password over HTTPS (SSL/TLS)
  - Client → Server: plaintext password (over TLS)
- Hash on the server with salt + slow algorithm
  - `slow_hash(password + salt)`
- Use sessions or tokens afterward
  - Cookies (`HttpOnly`, `Secure`)
  - Short-lived tokens

Secure Password Handling Model (2)

- Use a strong but slow password hashing algorithm:
  - Argon2id preferred
  - bcrypt acceptable
  - scrypt acceptable
  - Avoid fast hashes like SHA-256, SHA-1, or MD5 for passwords (Why?)
- For stronger protection, use MFA (Multi-Factor Authentication).

Salting Passwords

What is Salting?

- Adding a unique, random value (*salt*) to each password before hashing
- Stored together with the hash

How it works

- Compute hash: $\text{hash} = \text{Argon2id}(\text{password} + \text{salt})$
- Store: $(\text{salt}, \text{hash})$

Why use salting?

- Prevents identical passwords from having identical hashes
- Protects against rainbow table attacks
- Forces attackers to crack each password individually

Key Points

- Salt is **not secret**
- Must be **unique per user**
- Use a **secure random generator**
- Combine with slow hash functions (Argon2id, bcrypt, scrypt)

OPAQUE: Password Login Without Sending Passwords

Goal: authenticate using a password without sending the password or a reusable hash.

- OPAQUE is an **asymmetric PAKE** (password Authenticated Key Exchange) protocol
- Client knows the password
- Server stores an **encrypted credential**, not the password
- Login uses an **Oblivious Pseudo Random Function (OPRF)**

Core idea:

$$k = \mathrm{KDF}(\mathrm{OPRF}(\text{password}, \text{server secret}))$$

The server helps derive the login key, but never learns the password.

How OPAQUE Works

Registration

1. Client enters password
2. Client and server run OPRF
3. Client derives key: $k = \text{KDF}(\text{OPRF output})$
4. Client encrypts credential data using k
5. Server stores: $(\text{encrypted credential}, \text{public key})$

Login

1. Client repeats OPRF with password
2. Correct password recreates k
3. Client decrypts credential
4. Client and server prove possession and derive a session key